Conventional encryption principles, conventional encryption algorithms, cipher block modes of operation, location of encryption devices, key distribution approaches of messageauthentication, secure hash functions and hmac

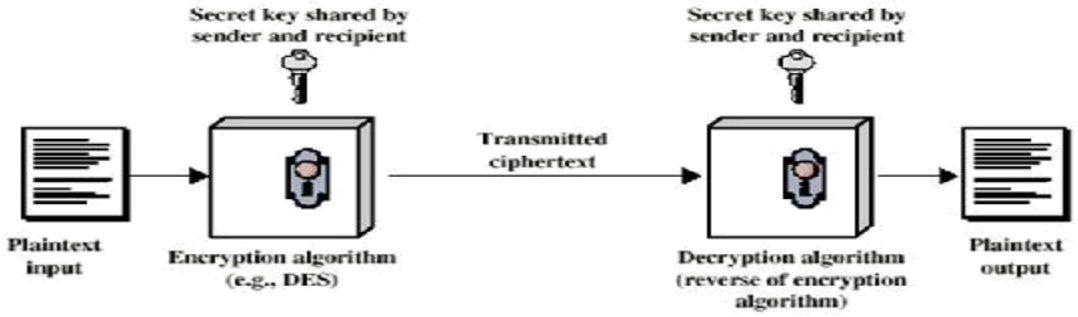
Conventional Encryption principles

A Symmetric encryption scheme has five ingredients

1. ***Plain Text***: This is the original message or data which is fed into the algorithm as input.

2. ***Encryption Algorithm***: This encryption algorithm performs various substitutions and transformations on the plain text.

3. ***Secret Key***: The key is another input to the algorithm. The substitutions and transformations performed by algorithm depend on the key.



Simplified Model of Conventional Encryption

4. ***Cipher Text*:** This is the scrambled (unreadable) message which is output of the encryption algorithm. This cipher text is dependent on plaintext and secret key. For a given plaintext, two different keys produce two different cipher texts.

5. ***Decryption Algorithm***: This is the reverse of encryption algorithm. It takes the cipher text and secret key as inputs and outputs the plain text.

Two main requirements are needed for secure use of conventional encryption:

(i). A strong encryption algorithm is needed. It is desirable that the algorithm should be in such a way that, even the attacker who knows the algorithm and has access to one or more cipher texts would be unable to decipher the cipher text or figure out the key.

(ii). The secret key must be distributed among the sender and receiver in a very secured way. If in any way the key is discovered and with the knowledge of algorithm, all communication using this key is readable.

Cryptography

A cipher is a secret method of writing, as by code. **Cryptography**, in a very broad sense, is the study of techniques related to aspects of information security. Hence cryptography is concerned with the writing (ciphering or encoding) and deciphering (decoding) of messages in secret code. Cryptographic systems are classified along three independent dimensions:

- ***The type of operations used for performing plaintext to ciphertext***

  All the encryption algorithms make use of two general principles; substitution and transposition through which plaintext elements are rearranged. Important thing is that no information should be lost.

- ***The number of keys used***

  If single key is used by both sender and receiver, it is called symmetric, single-key, secret-key or conventional encryption. If sender and receiver each use a different key, then it is called asymmetric, two-key or public-key encryption.

- ***The way in which plaintext is processed***

  A block cipher process the input as blocks of elements and generated an output block for each input block. Stream cipher processes the input elements continuously, producing output one element at a time as it goes along.

Cryptanalysis

The process of attempting to discover the plaintext or key is known as cryptanalysis. It is very difficult when only the cipher text is available to the attacker as in some cases even the encryption algorithm is not known. The most common attack under these circumstances is brute-force approach of trying all the possible keys. This attack is made impractical when the key size is considerably large. The table below gives an idea on types of attacks on encrypted messages.

Type of Attack	Known to Cryptanalyst
Ciphertext only	• Encryption algorithm • Ciphertext to be decoded
Known plaintext	• Encryption algorithm • Ciphertext to be decoded • One or more plaintext-ciphertext pairs formed with the secret key
Chosen plaintext	• Encryption algorithm • Ciphertext to be decoded • Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen ciphertext	• Encryption algorithm • Ciphertext to be decoded • Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen text	• Encryption algorithm • Ciphertext to be decoded • Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key • Purported ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

Cryptology covers both cryptography and cryptanalysis. *Cryptology* is a constantly evolving science; ciphers are invented and, given time, are almost certainly breakable. *Cryptanalysis* is the best way to understand the subject of cryptology. *Cryptographers* are constantly searching for the perfect security system, a system that is both fast and hard and a system that encrypts quickly but is hard or impossible to break. *Cryptanalysts* are always looking for ways to break the security provided by a cryptographic system, mostly though

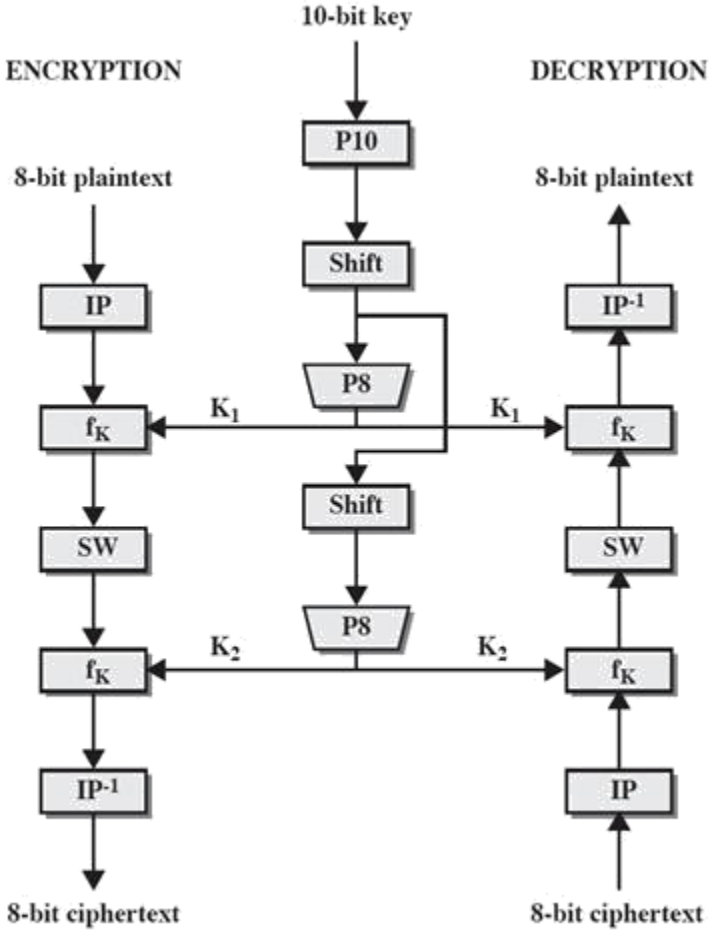
mathematical understanding of the cipher structure.

Cryptography can be defined as the conversion of data into a scrambled code that can be deciphered and sent across a public or a private network.

- A ***Ciphertext-only attack*** is an attack with an attempt to decrypt ciphertext when only the ciphertext itself is available.
- A ***Known-plaintext attack*** is an attack in which an individual has the plaintext samples and its encrypted version (ciphertext) thereby allowing him to use both to reveal further secret information like the key
- A ***Chosen- plaintext attack*** involves the cryptanalyst be able to define his own plaintext, feed it into the cipher and analyze the resulting ciphertext.
- A ***Chosen-ciphertext attack*** is one, where attacker has several pairs of plaintext-ciphertext and ciphertext chosen by the attacker.

Simplified DES

S-DES is a reduced version of the DES algorithm. It has similar properties to DES but deals with a much smaller block and key size (operates on 8-bit message blocks with a 10-bit key). The S-DES decryption algorithm takes an 8-bit block of ciphertext and the same 10-bit key used to produce that ciphertext as input and produces the original 8-bit block of plaintext. S-DES scheme is shown below:



Simplified DES Scheme

The encryption algorithm involves five functions: and initial permuatation(IP), a complex function labeled f_k, which involves both permutations and substitution operations and depends on a key input, a single permutation function (SW) that switches the two halves of the data, the function f_k again and finally a permutation function that is inverse of the IP i.e. IP^{-1}.

As shown in figure, the function f_k takes the data from encryption function along with 8-bit key. The key is choosen to be 10-bit length from which two 8-bit subkeys are generated. The initial 10-bit key is subjected to a permutation (P10) followed by a shift operation. The output of this shift operation then passes through a permutation function that produces an 8-bit output (P8) for the first key (k_1) and also feeds into another shiftand another instance of P8 to produce the second subkey (k_2). The encryption algorithm can be written as:

$$\textbf{Ciphertext} = IP^{-1}(f_{k2}(SW(f_{k1}(IP(\text{plaintext})))))$$

Where K_1 = P8(shift(p10(key)))

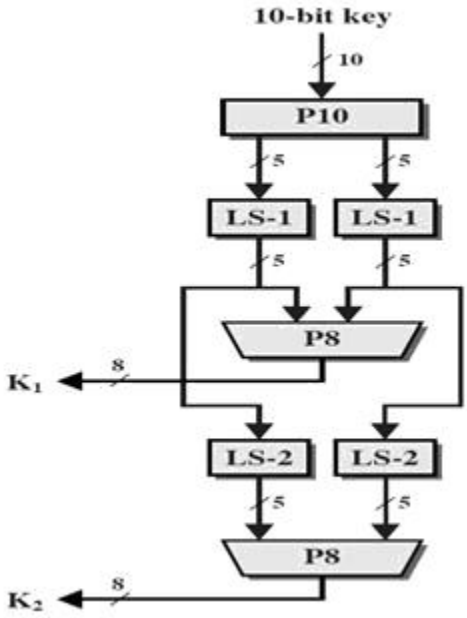
K_2 = P8(shift(shift(p10(key))))

Decryption is also shown in the above figure and can be given as:

$$\textbf{Plaintext} = IP^{-1}(f_{k1}(SW(f_{k2}(IP(\text{ciphertext})))))$$

Key Generation:

The key generation process is shown below:



As shown above, a 10-bit key shared between sender and receiver is used and fist passed through a permutation P10.

The Switch Function:

This function interchanges the left and right 4 bits so that the second instance of $\mathbf{f_K}$ operates on a different 4 bits. For second instance all other parameters remain same, but the key is K_2. The S-boxes operates as follows:- The first and fourth input bits are treated as 2-bit numbers that specify a row of the S-box, and the second and third input bits specify a column of S-box. The entry in that row and column in base2 is the 2-bit output.

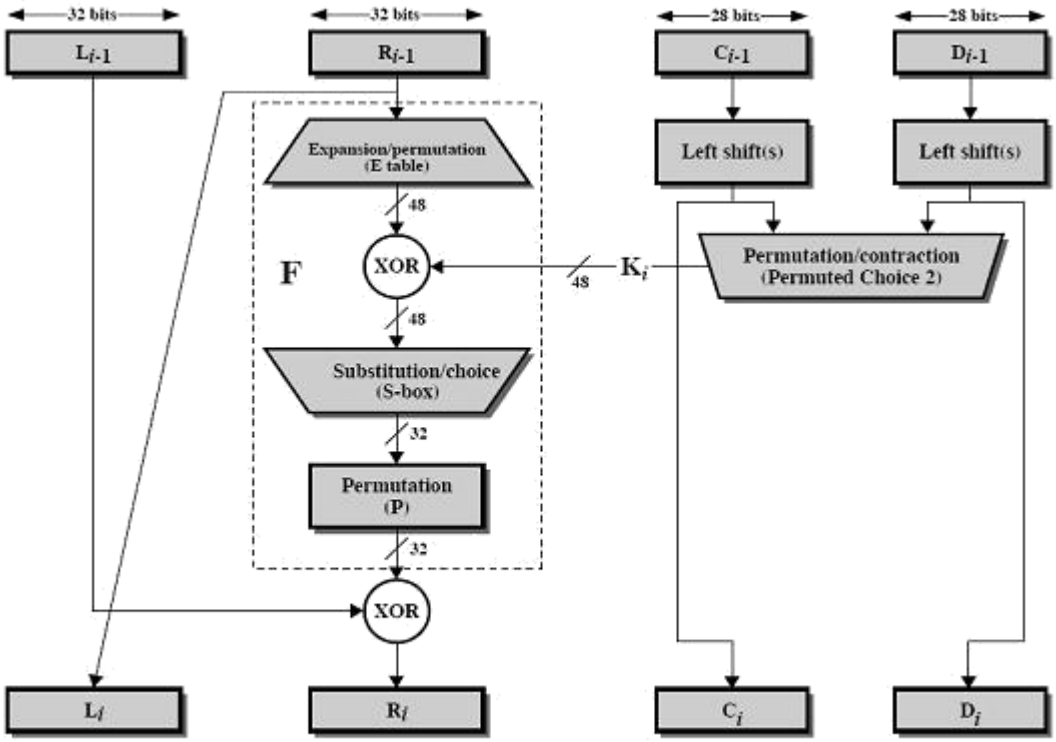
Data Encryption Standard

In 1974, IBM proposed "Lucifer", an encryption algorithm using 64-bit keys. Two years later (1977), NBS (now NIST) in consultation with NSA made a modified version of that algorithm into a standard. DES uses the two basic techniques of cryptography - confusion and diffusion. At the simplest level, diffusion is achieved through numerous permutations and confusion is achieved through the XOR operation and the S-Boxes. This is also called an S-P network The DES encryption scheme can be explained by the following figure

The plain text is 64 bits in length and the key in 56 bits in length. Longer plain text amounts are processed in 64-bit blocks. The main phases in the left hand side of the above figure i.e. processing of the plain text are,

- **Initial Permutation (IP):** The plaintext block undergoes an initial permutation. 64 bits of the block are permuted.
- **A Complex Transformation:** 64 bit permuted block undergoes 16 rounds of complex transformation. Subkeys are used in each of the 16 iterations.
- **32-bit swap:** The output of 16th round consists of 64bits that are a function of input plain text and key.32 bit left and right halves of this output is swapped.
- **Inverse Initial Permutation (IP^{-1}):** The 64 bit output undergoes a permutation that is inverse of the initial permutation.
- **A Complex Transformation:** 64 bit permuted block undergoes 16 rounds of complex transformation. Subkeys are used in each of the 16 iterations.
- **32-bit swap:** The output of 16th round consists of 64bits that are a function of input plain text and key.32 bit left and right halves of this output is swapped.
- **Inverse Initial Permutation (IP^{-1}):** The 64 bit output undergoes a permutation that is inverse of the initial permutation.

The following figure shows a closer view of algorithms for a single iteration. The 64bit permuted input passes through 16 iterations, producing an intermediate 64-bit value at the conclusion of each iteration.



Single Round of DES Algorithm

The left and right halves of each 64 bit intermediate value are treated as separated 32-bit quantities labeled L (left) and R (Right). The overall processing at each iteration is given by following steps, which form one round in an S-P network.

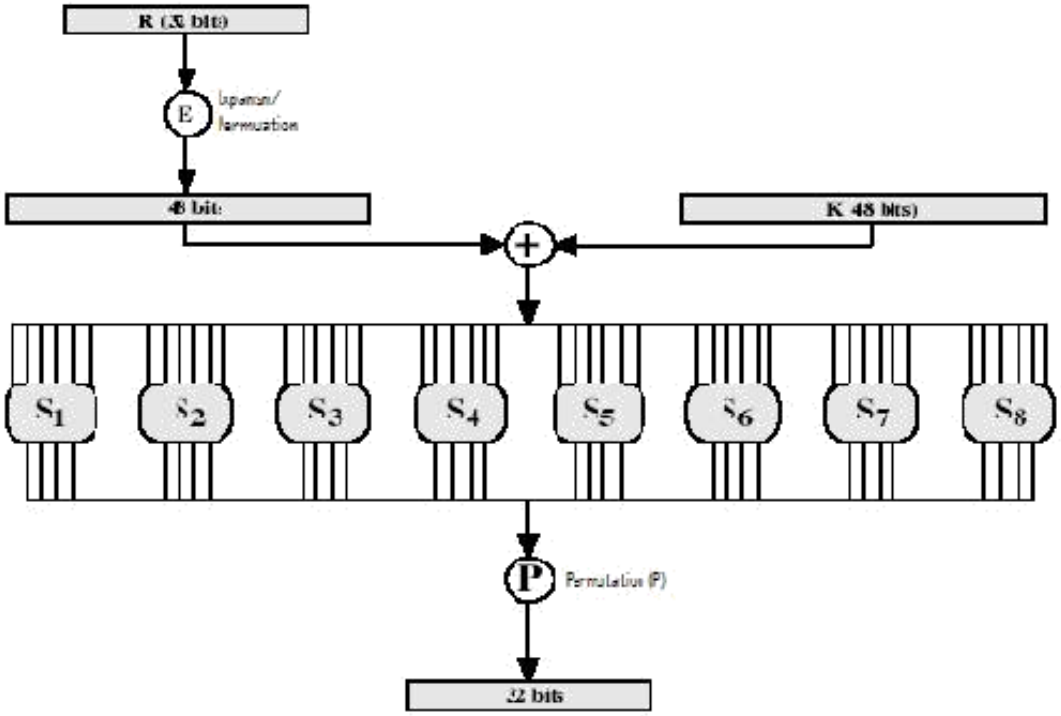
Li = Ri-1.

Ri = L i-1 $\oplus$ F(R i-1, Ki)

Where Function F can be described as **P(S(E(R(i-1)) □ K(i)))**

The left hand output of an iteration (Li) is equal to the right hand input to that iteration Ri-1. The right hand output Ri is exclusive OR of Li-1 and a complex function F of Ri-1 and Ki. The fucntion F can be depicted by the following figure. S1, S2 S8 represent the "S-boxes" , which maps each combination of 48 input bits into a particular 32 bit pattern. For the generation of subkey of length 48 bits, a 56bit key is used which is first passed through a permutation funciton and then halved to get two 28 bit quantities labeled C0 and D0. At each iteration, these two C and D are subjected to a circular left shift or rotation of 1 or 2 bits. These shifted values serve as input to the next iteration and also to another permutation function which produces a 48-bit output. This output is fed as input to function

F(R $_{i-1}$, Ki).



The first and last bits of the input to the box Si form a 2-bit binary number to select one of four substitutions defined by the four rows in the table for Si. The middle 4-bits select a particular column. The decimal value in the cell selected by the row and column is converted to its 4-bit representation to produce the output.

The substitution consists of a set of eight S-boxes, each of which accepts 6 bits as input and produces 4 bits as output. The process of decryption with DES is essentially the same as the encryption process: no different algorithm is used. The ciphertext is used as input to the DES algorithm and the keys are used in the reverse order i.e. K16 in the first iteration, K15 on the second iteration and so on until k1 is used on the sixteenth and last iteration.

Strength of DES:

Avalanche Effect: An effect in DES and other secret key ciphers where each small change in plaintext implies that somewhere around half the ciphertext changes. The avalanche effect makes it harder to successfully cryptanalyze the ciphertext. DES exhibits a strong Avalanche effect.

Concern about the strength of DES falls into two categories i.e. strength of algorithm itself and use of 56-bit key. Though many attempts were made over the years to find and exploit weaknesses in the algorithm, none of them were successful in discovering any fatal weakness in DES. A serious concern is with the key size as the time passed the security in DES became getting compromised by the advent of supercomputers which succeeded in breaking the DES quickly using a brute-force attack. If the only form of attack that could be made on an encryption algorithm is brute force, the way of countering it is obviously using long keys. If a key of size 128 bits is used, it takes approximately 10^{18} years to break the code making the algorithm unbreakable by brute-force approach.

The two analytical attacks on DES are Differential cryptanalysis and Linear cryptanalysis. Both make use of Known plaintext-ciphertext pairs and try to attack the round structure and the S-Boxes. Recent advancements showed that using Differential cryptanalysis, DES can be broken using 2^{47} plaintext-ciphertext pairs and for linear cryptanalysis, the number is even reduced to 2^{41}.

Triple DES

The first answer to problems of DES is an algorithm called Double DES which includes double encryption with two keys. It increases the key size to 112 bits, which seems to be secure. But, there are some problems associated with this approach.

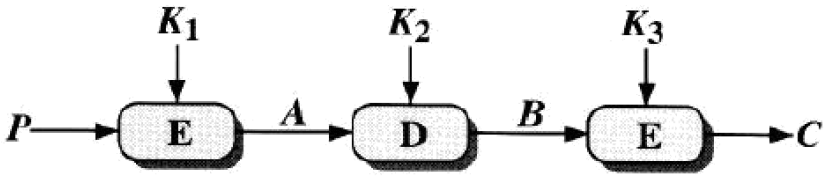
issue of reduction to single stage:

In other words, could there be a key K3 such that $E_{K2}(E_{K21}(P)) = E_{K3}(P)$?

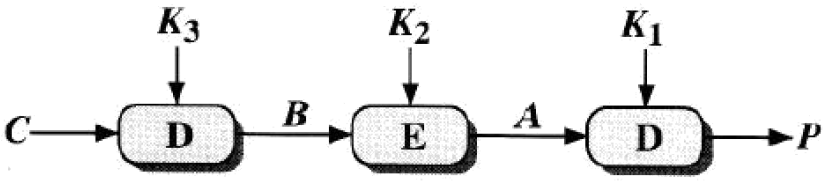
"meet-in-the-middle" attack:

- Works when given a known (P,C) pair
- since $\mathbf{X = E_{K1}(P) = D_{K2}(C)}$
- attack by encrypting P with all 2^{56} keys $K_1$and store
- then decrypt C with all possible 2^{56} keys K_2 and match X value

Triple DES was the answer to many of the shortcomings of DES. Since it is based on the DES algorithm, it is very easy to modify existing software to use Triple DES. 3DES was developed in 1999 by IBM – by a team led by Walter Tuchman. 3DES prevents a meet-in-the-middle attack. 3DES has a 168-bit key and enciphers blocks of 64 bits. It also has the advantage of proven reliability and a longer key length that eliminates many of the shortcut attacks that can be used to reduce the amount of time it takes to break DES. 3DES uses three keys and three executions of the DES algorithm. The function follows an encrypt-decrypt-encrypt (EDE) sequence.



(a) Encryption



(b) Decryption

$$C = E_{K3}[D_{K2}[E_{K1}[P]]]$$

Where C= ciphertext, P= plaintext and
$E_K[X]$ = encryption of X using key K
$D_K[Y]$ = decryption of Y using key K
Decryption is simply the same operation with the keys reversed

$$P = D_{K1}[E_{K2}[D_{K3}[C]]]$$

Triple DES runs three times slower than standard DES, but is much more secure if used properly. With three distinct keys, TDEA has an effective key length of 168 bits making it a formidable algorithm. As the underlying algorithm is DEA, it offers the same resistance to cryptanalysis as is DEA.
Triple DES can be done using 2 keys or 3 keys.

AES is a symmetric block cipher that is intended to replace DES as the approved standard for a wide range of applications. The drawbacks of 3DES being it is very slow and also it uses 64-bit block size same as DES. For reasons of both efficiency and security, a larger key size is desirable. So, NIST (National Institute of Standards and Technology) has called for proposals for a new AES, which should have security strength equal to or better than 3DES and significantly, improved efficiency. NIST specified that AES must be a symmetric block cipher with a block length of 128 bits and support for key lengths of 128, 192, and 256 bits.

Out of all the algorithms that were submitted, five were shortlisted and upon final evaluation, NIST selected **Rijndael** as the proposed AES algorithm. The two researchers who developed and submitted Rijndael for the AES are both cryptographers from Belgium: **Dr. Joan Daemen and Dr. Vincent Rijmen.**

AES Evaluation:

- There are three main categories of criteria used by NIST to evaluate potential candidates.
- *Security:* Resistance to cryptanalysis, soundness of math, randomness of output, etc
- *Cost:* Computational efficiency (speed), Memory requirements
- *Algorithm/Implementation Characteristics:* Flexibility, hardware and software suitability, algorithm simplicity

Simplified AES

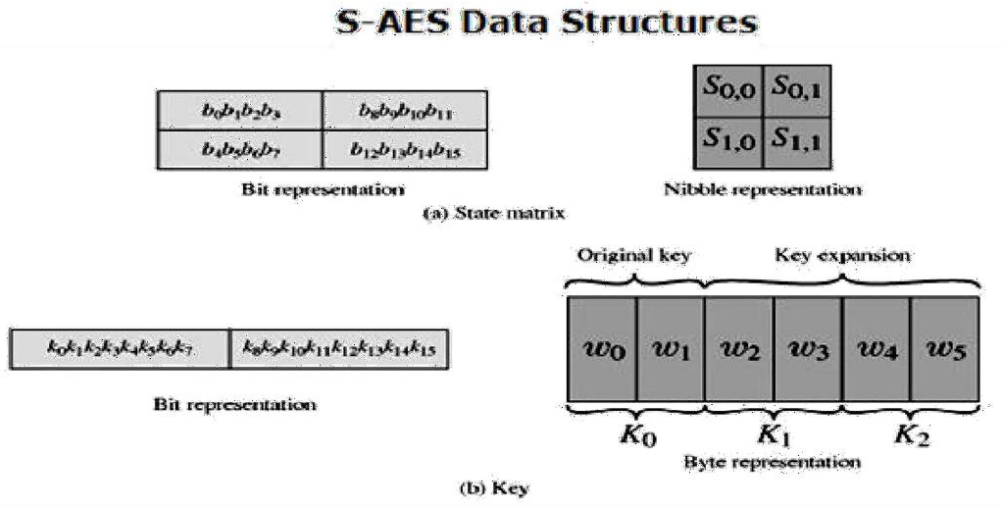
The encryption algorithm takes a 16-bit block of plaintext as input and a 16-bit key and produces a 16-bit block of ciphertext as output. The S-AES decryption algorithm takes a 16-bit block of ciphertext and the same 16-bit key used to produce that ciphertext as input and produces the original 16-bit block of plaintext as output. The encryption algorithm involves the use of four different functions, or transformations: add key (A_K) nibble substitution (NS), shift row (SR), and mix column (MC).

The encryption algorithm can be expressed as:

$$A_{K_2} \circ SR \circ NS \circ A_{K_1} \circ MC \circ SR \circ NS \circ A_{K_0}$$, so that A_{K0} is applied first.

The encryption algorithm is organized into three rounds. Round 0 is simply an add key round; round 1 is a full round of four functions; and round 2 contains only 3 functions. Each round includes the add key function, which makes use of 16 bits of key. The initial 16-bit key is expanded to 48 bits, so that each round uses a distinct 16-bit round key. S- AES encryption and decryption scheme is shown below.

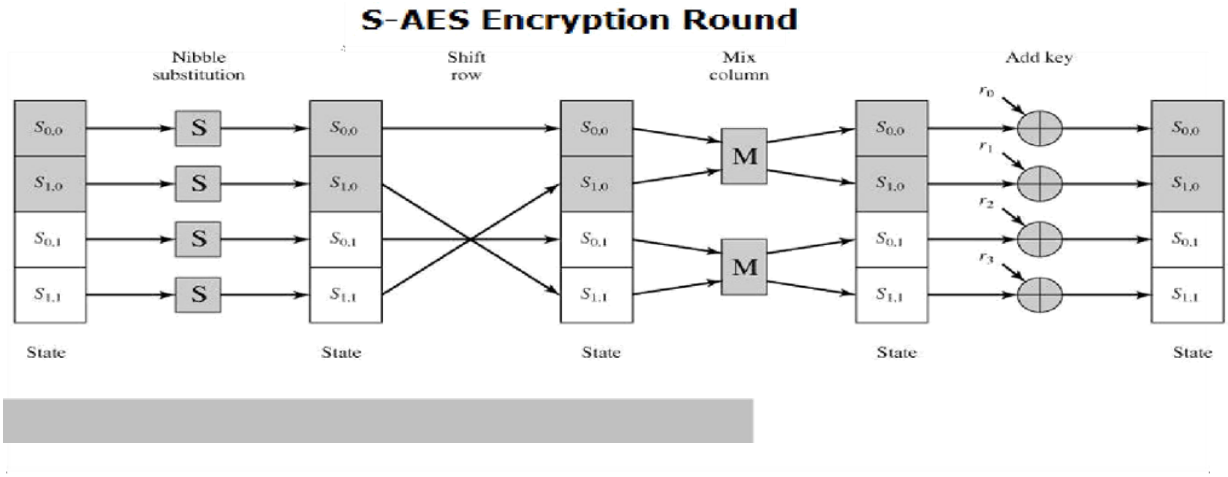
Each function operates on a 16-bit state, treated as a 2 x 2 matrix of nibbles, where one nibble equals 4 bits. The initial value of the state matrix is the 16-bit plaintext; the state matrix is modified by each subsequent function in the encryption process, producing after the last function the 16-bit ciphertext. The following figure shows the ordering of nibbles within the matrix is by column. So, for example, the first eight bits of a 16-bit plaintext input to the encryption cipher occupy the first column of the matrix, and the second eight bits occupy the second column. The 16-bit key is similarly organized, but it is somewhat more convenient to view the key as two bytes rather than four nibbles The expanded key of 48 bits is treated as three round keys, whose bits are labelled as follows: $K_0 = k_0...k_{15}$; $K_1 = k_{16}...k_{31}$; $K_2 = k_{32}...k_{47}$.



The following figure shows the essential elements of a full round of S-AES. The decryption as shown above can be given as:

$$A_{K_0} \circ INS \circ ISR \circ IMC \circ A_{K_1} \circ INS \circ ISR \circ A_{K_2}$$

in which three of the functions have a corresponding inverse function: inverse nibble substitution (INS), inverse shift row (ISR), and inverse mix column (IMC).



S-AES Encryption and Decryption

The individual functions that are part of the encryption algorithm are given below.

Add Key

A	4	⊕	2	5	=	8	1
7	9		D	5		A	C
state matrix			key				

The add key function consists of the bitwise XOR of the 16-bit state matrix and the 16-bit round key. As shown in the above example, it can also be viewed as a nibble-wise or bitwise operation. The inverse of the add key function is identical to the add key function, because the XOR operation is its own inverse.

Nibble Substitution

The nibble substitution function is a simple table lookup. AES defines a 4 x 4 matrix of nibble values, called an S-box that contains a permutation of all possible 4-bit values. Each individual nibble of the state matrix is mapped into a new nibble in the following way: The leftmost 2 bits of the nibble are used as a row value and the rightmost 2 bits are used as a column value. These row and column values serve as indexes into the S-box to select a unique 4-bit output value. For example, the hexadecimal value A references row 2, column 2 of the S-box, which contains the value 0. Accordingly, the value A is mapped into the value 0.

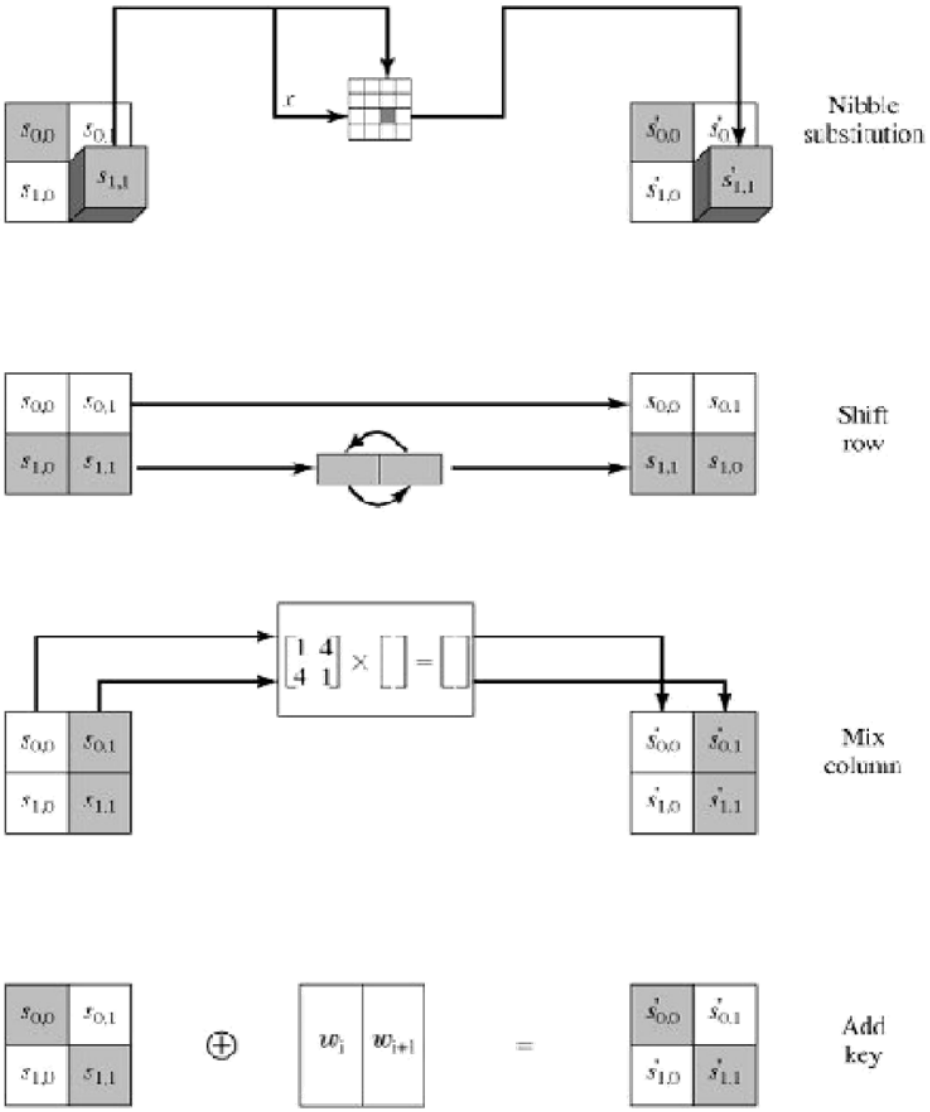
For the example, after nibble substitution, the output is

8	1
A	C

→

6	4
0	C

S-AES Transformations



Shift Row

The shift row function performs a one-nibble circular shift of the second row of the state matrix; the first row is not altered. Our example is shown below:

6	4
0	C

→

6	4
C	0

The inverse shift row function is identical to the shift row function, because it shifts the second row back to its original position.

Mix Column

The mix column function operates on each column individually. Each nibble of a column is mapped into a new value that is a function of both nibbles in that column. The transformation can be defined by the following matrix multiplication on the state matrix.

$$\begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} S_{0,0} & S_{0,1} \\ S_{1,0} & S_{1,1} \end{bmatrix} = \begin{bmatrix} S'_{0,0} & S'_{0,1} \\ S'_{1,0} & S'_{1,1} \end{bmatrix}$$

Where arithmetic is performed in GF(2^4), and the symbol · refers to multiplication in GF(2^4). The example is shown below:

$$\begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} 6 & 4 \\ C & 0 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 7 & 3 \end{bmatrix}$$

The inverse mix column function is defined as follows:

$$\begin{bmatrix} 9 & 2 \\ 2 & 9 \end{bmatrix} \begin{bmatrix} S_{0,0} & S_{0,1} \\ S_{1,0} & S_{1,1} \end{bmatrix} = \begin{bmatrix} S'_{0,0} & S'_{0,1} \\ S'_{1,0} & S'_{1,1} \end{bmatrix}$$

Key Expansion

For key expansion, the 16 bits of the initial key are grouped into a row of two 8-bit words. The following figure shows the expansion into 6 words, by the calculation of 4 new words from the initial 2 words. The algorithm is as follows:

$$w_2 = w_0 \oplus g(w_1) = w_0 \oplus \text{RCON}(1) \oplus \text{SubNib}(\text{RotNib}(w_1))$$

$$w_3 = w_2 \oplus w_1$$

$$w_4 = w_2 \oplus g(w_3) = w_2 \oplus \text{RCON}(2) \oplus \text{SubNib}(\text{RotNib}(w_3))$$

$$w_5 = w_4 \oplus w_3$$

RCON is a round constant, defined as follows: **RC[i] = x^{i+2}, so that RC[1]=x^3=1000** and **RC[2]=x^4 mod (x^4 + x + 1) = x + 1 = 0011**. RC[i] forms the leftmost nibble of a byte, with the rightmost nibble being all zeros. Thus, RCON(1) = 10000000 and RCON(2) = 00110000.

For example, suppose the key is 2D55 = 0010 1101 0101 0101 = w_0w_1. Then,

$$w_2 = 00101101 \oplus 10000000 \oplus \text{SubNib}(01010101)$$

$$= 00101101 \oplus 10000000 \oplus 00010001 = 10111100$$

$$w_3 = 10111100 \oplus 01010101 = 11101001$$

$$w_4 = 10111110 \oplus 00110000 \oplus \text{SubNib}(10011110)$$

$$= 10111100 \oplus 00110000 \oplus 00101111 = 10100011$$

$$w_5 = 10100011 \oplus 11101001 = 01001010$$

The S-Box

The S-box is constructed as follows:

- Initialize the S-box with the nibble values in ascending sequence row by row. The first row contains the hexadecimal values 0, 1, 2, 3; the second row contains 4, 5, 6, 7; and so on. Thus, the value of the nibble at row i, column j is 4i + j.
- Treat each nibble as an element of the finite field GF(2^4) modulo x^4 +x + 1. Each nibble $a_0a_1a_2a_3$ represents a polynomial of degree 3.
- Map each byte in the S-box to its multiplicative inverse in the finite field GF(2^4) modulo x^4 + x + 1; the value 0 is mapped to itself.
- Consider that each byte in the S-box consists of 4 bits labeled (b_0, b_1, b_2, b_3). Apply the following transformation to each bit of each byte in the S-box: The AES standard depicts this transformation in matrix form as follows:

$$\begin{bmatrix} b_0' \\ b_1' \\ b_2' \\ b_3' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

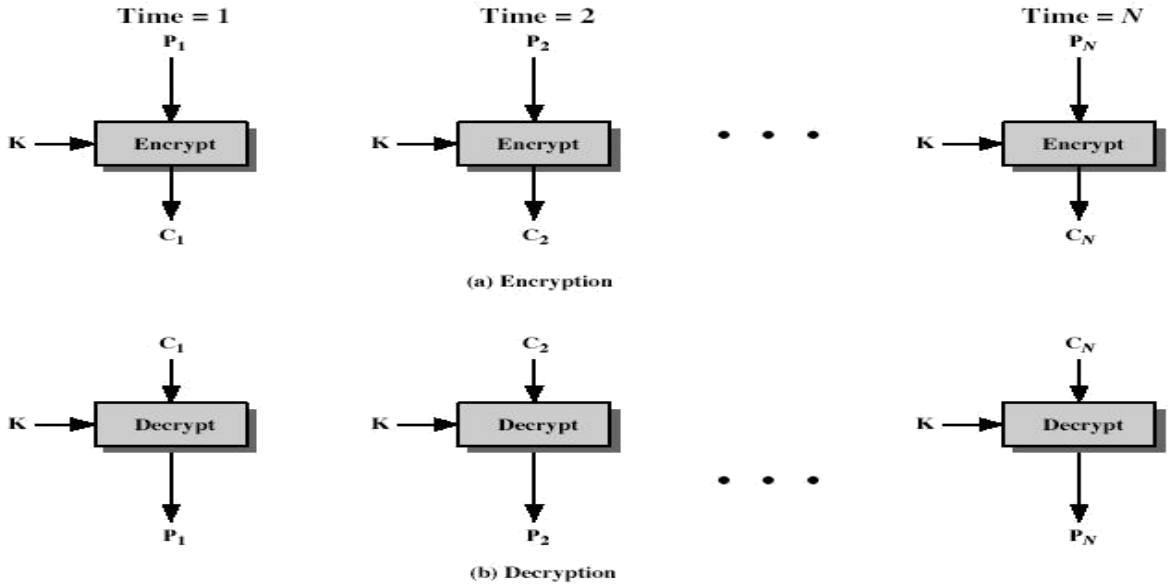
The prime (') indicates that the variable is to be updated by the value on the right. Remember that addition and multiplication are being calculated modulo 2.

Cipher Block modes of Operation

To apply a block cipher in a variety of applications, four "modes of operation" have been defined by NIST (FIPS 81). The four modes are intended to cover virtually all the possible applications of encryption for which a block cipher could be used. As new applications and requirements have appeared, NIST has expanded the list of recommended modes to five in Special Publication 800-38A. These modes are intended for use with any symmetric block cipher, including triple DES and AES.

Electronic Codebook Book (ECB)

The simplest mode is the electronic codebook (ECB) mode, in which plaintext is handled one block at a time and each block of plaintext is encrypted using the same key. *ECB is the simplest of the modes, and is used when only a single block of info needs to be sent.*



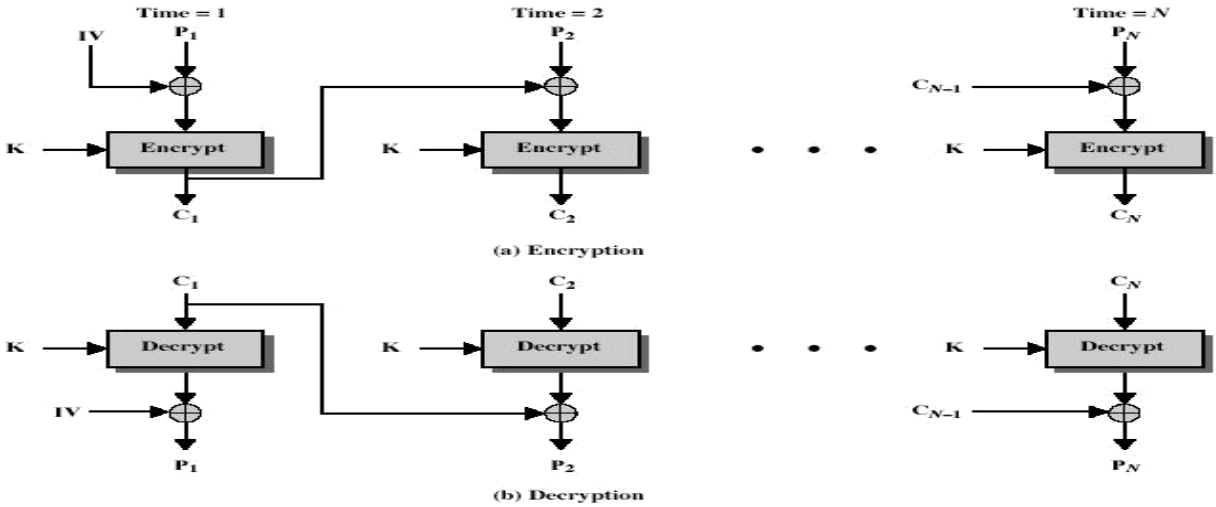
Break the plaintext into 64-bit blocks and encrypt each of them with the same key. The last block should be padded to 64-bit if it is shorter. Same block and same key always yields same cipher block. Each block is a value which is substituted, like a codebook, hence the name Electronic Code Book. Each block is encoded independently of the other blocks.

$$C_i = DES_{K1}(P_i)$$

ECB is not appropriate for any quantity of data, since repetitions can be seen, esp. with graphics, and because the blocks can be shuffled/inserted without affecting the en/decryption of each block. Its main use is to send one or a very few blocks, eg a session encryption key.

Cipher Block Chaining Mode (CBC)

To overcome the problems of repetitions and order independence in ECB, want some way of making the ciphertext dependent on **all** blocks before it. This is what CBC gives us, by combining the previous ciphertext block with the current message block before encrypting. To start the process, use an Initial Value (IV), which is usually well known (often all 0's), or otherwise is sent, ECB encrypted, just before starting CBC use.



All cipher blocks will be chained so that if one is modified, the ciphertext cannot be decrypted correctly. Each plaintext block is XORed with the previous cipher block before encryption, hence the name CBC. The first plaintext block is XORed with an initialization vector IV, which is to be protected securely, (e.g., send it encrypted in ECB mode).

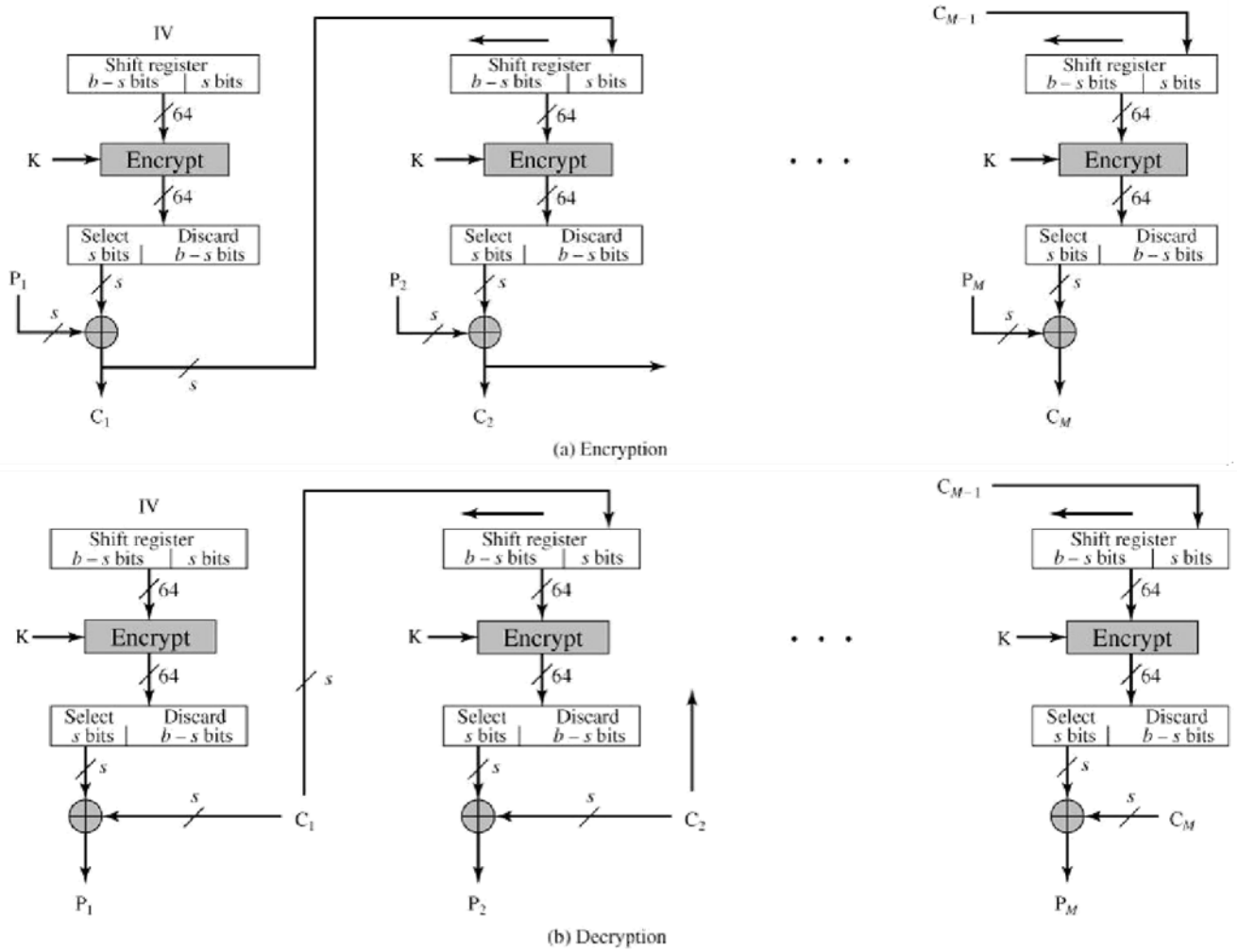
$$C_i = DES_{K1}(P_i \text{ XOR } C_{i-1})$$

CBC is the block mode generally used. The chaining provides an avalanche effect, which means the encrypted message cannot be changed or rearranged without totally destroying the subsequent data. However there is the issue of ensuring that the IV is either fixed or sent encrypted in ECB mode to stop attacks on 1st block.\

r Feed Back Mode (CFB)

If the data is only available a bit/byte at a time (eg. terminal session, sensor value etc), then must use some other approach to encrypting it, so as not to delay the info. it is possible to convert DES into a stream cipher, using either the cipher feedback (CFB) or the output feedback mode. A stream cipher eliminates the need to pad a message to be an integral number of blocks. It also can operate in real time. Thus, if a character stream is being transmitted, each character can be encrypted and transmitted immediately using a character-oriented stream cipher.

One desirable property of a stream cipher is that the ciphertext be of the same length as plaintext. Thus, if 8-bit characters are being transmitted, each character should beencrypted to produce a cipher text output of 8 bits. If more than 8 bits are produced, transmission capacity is wasted.



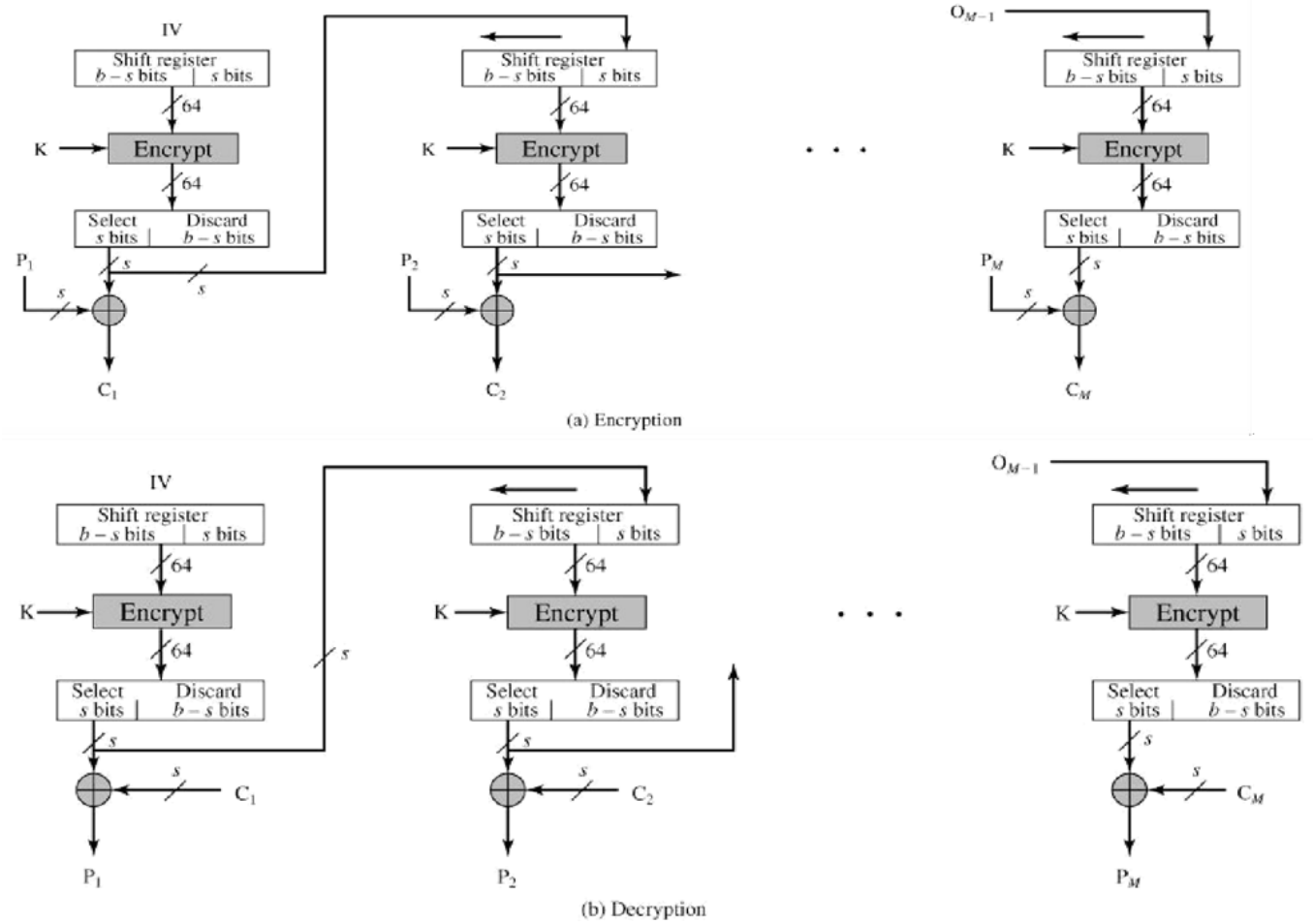
The input to the encryption function is a b-bit shift register that is initially set to some initialization vector (IV). The leftmost (most significant) s bits of the output of the encryption function are XORed with the first segment of plaintext P_1 to produce the first unit of ciphertext C_1, which is then transmitted. In addition, the contents of the shift register are shifted left by s bits and C_1 is placed in the rightmost (least significant) s bits of the shift register. This process continues until all plaintext units have been encrypted. For decryption, the same scheme is used, except that the received ciphertext unit is XORed with the output of the encryption function to produce the plaintext unit. Note that it is the *encryption* function that is used, not the decryption function.

$$\mathbf{C_i = P_i \text{ XOR } DES_{K1}(C_{i-1})}$$

CFB is the usual stream mode. As long as can keep up with the input, doing encryptions every 8 bytes. A possible problem is that if its used over a "noisy" link, then any corrupted bit will destroy values in the current and next blocks (since the current block feeds as input to create the random bits for the next). So either must use over a reliable network transport layer (pretty usual) or use OFB.

Output Feedback Mode (OFB)

The output feedback (OFB) mode is similar in structure to that of CFB. It is the output of the encryption function that is fed back to the shift register in OFB, whereas in CFB the ciphertext unit is fed back to the shift register.



Keystream is independent of the data and can be computed in advance.

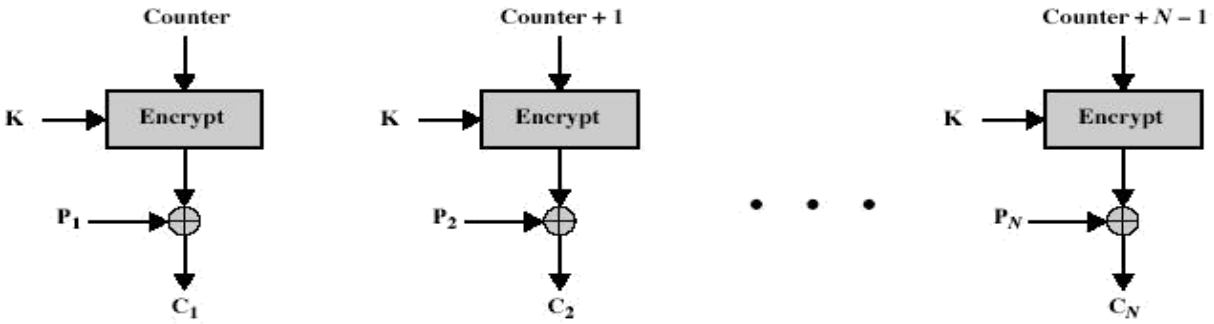
$$C_i = P_i \text{ XOR } O_i \; O_i$$
$$= DES_{K1}(O_{i-1})$$

Here the generation of the "random" bits is independent of the message being encrypted. The advantage is that firstly, they can be computed in advance, good for bursty traffic, and secondly, any bit error only affects a single bit. Thus this is good for noisy links (eg satellite TV transmissions etc). The disadvantage of OFB is that it is more vulnerable to a message stream modification attack than is CFB.

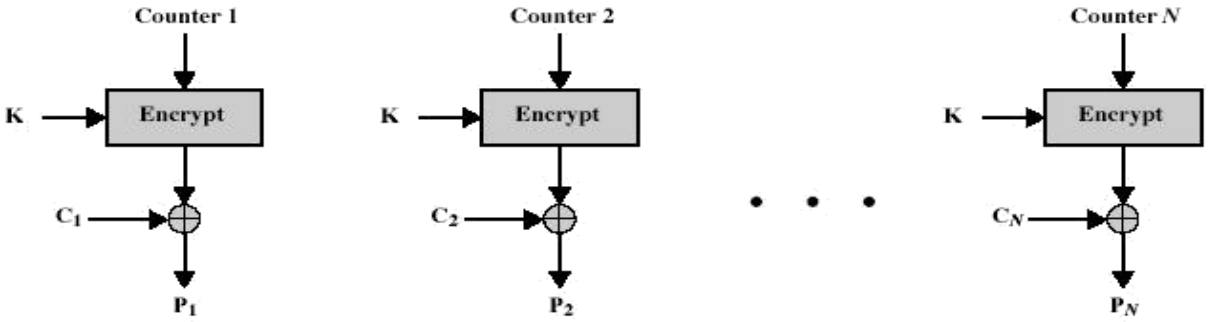
Counter Mode (CTR)

The Counter (CTR) mode is a variant of OFB, but which encrypts a counter value (hence name). Although it was proposed many years before, it has only recently been standardized for use with AES along with the other existing 4 modes. It is being used with applications in ATM (asynchronous transfer mode) network security and IPSec (IP security).

All modes of operations except ECB make random access to the file impossible: to access data at the end of the file one has to decrypt everything. Plaintext is not encrypted directly. IV plus a constant is encrypted and the resulting ciphertext is XORed with the plaintext – add 1 to IV in each step.



(a) Encryption



(b) Decryption

If the same IV is used twice with the same key, then cryptanalyst may XOR the ciphers to get the XOR of the plaintexts –this could be used in an attack. A counter, equal to the plaintext block size is used. The only requirement stated in SP 800-38A is that the counter value must be different for each plaintext block that is encrypted. Typically the counter is initialized to some value and then incremented by 1 for each subsequent block.

CTR mode has a number of advantages in parallel h/w & s/w efficiency, can preprocess the output values in advance of needing to encrypt, can get random access to encrypted data blocks, and is simple. But like OFB have issue of not reusing the same key + counter value.

27